

LYRA-CPP TX EXECUTION PLAN — Step 14 → Phase 3

Branch: tx-rebuild

Reference tree: D:\sdrprojects\OpenHPSDR-Thetis-2.10.3.13\Project Files\Source\

- ChannelMaster\networkproto1.c — HL2 P1 wire layer
- ChannelMaster\network.c — sendPacket + WriteUDPFrame + KeepAliveLoop
- ChannelMaster\network.h — _radionet struct + globals
- ChannelMaster\netInterface.c — create_rnet, thread spawn handshake
- ChannelMaster\cmaster.c, cmsetup.c — WDSP TXA channel setup
- Console\radio.cs — high-level WDSP TXA setter ordering
- Console\console.cs — MOX state machine (chkMOX_CheckedChanged2)
- wdsp*.c — WDSP DSP source (port-with-attribution per CLAUDE.md §2)

Created: 2026-06-06 by operator directive.

Owner: N8SDR. Co-author trailer on every commit: Co-Authored-By: Claude Opus 4.7 (1M context)

■ RELATIONSHIP TO STEP14_PLAN.md

This document is **ADDED** to the existing docs/architecture/STEP14_PLAN.md — not a replacement. The two docs stack:

Doc	Role
docs/architecture/STEP14_PLAN.md	Engineering blueprint. What each of Stages 1-10 technically does — code structure, reference behaviors to mirror, files touched, expected wire effects. Pre-existing, unchanged. The "WHAT."
docs/EXECUTION_PLAN.md *(this file)*	Operating discipline + progress tracking. Locked rules, pre-write verification protocol, 2-audit gate, §3.9 sign-off registry, per-stage checklists, STOP-AND-ASK queue, verification log. The "HOW we execute + what gates each step."

For any given stage:

1. Read STEP14_PLAN.md §Stage N — the engineering spec.
2. Read this file's §Stage N checklist — the operational gates.
3. Follow the Pre-Write Protocol below → write code → Audit #1 → Audit #2 → mark checkbox → commit → re-sync DOCX/PDF.

No checkbox in this file is marked complete until BOTH the engineering spec (STEP14_PLAN.md §Stage N) is satisfied AND the 2-audit gate (Rule 6 below) is satisfied. Bench-passing alone is necessary but not sufficient.

This file is the answer to two operator questions:

- **"What happens before any code is written?"** → Pre-Write Protocol (Verification Protocol §1-6) + Rule 3 (read reference twice) + Rule 4 (STOP-AND-ASK if unsure).
- **"What happens before anything is confirmed as PARITY-LIKE-REFERENCE?"** → 2-audit gate (Rule 6) + Verification Protocol §10-12 + (for bench-gated stages) operator bench sign-off §14-15.

■ LOCKED OPERATING RULES — READ AT START OF EVERY SESSION

These rules supersede any prior pattern. Violations cost real session time and operator trust. Every session begins by reading this section.

This section consolidates the full Lyra-cpp operating rules:

- **Rules 1-22** — foundational rules, effective 2026-06-04 (prior locked set, originally docs/RULES.md which is local-only and now superseded by this section as the canonical source).
- **Rule 23** — Amendment 1, parity checkpoint methodology (2026-06-05).
- **Rule 24** — Always verify against reference (locked 2026-06-05).
- **Rule 2 Amendment** — Signed sign-off cells exempted (locked 2026-06-05).
- **Rules 25-30** — 2026-06-06 session additions (audit-driven discipline lock-in).

Section A. Foundational

Rule 1 — Reference-parity Lyra implements the working reference's architecture (Thetis 2.10.3.13) and mirrors its measurable behaviour. Variations from the reference require explicit Rick + Claude agreement BEFORE any code is written, and are recorded as declared deviations in the design ledger. Silent variations are forbidden.

Rule 2 — Attribution discipline Lyra is an independent C++23 / Qt6 implementation, not a port or fork. Mentions of the working reference (Thetis 2.10.3.13) and other open-source SDR applications used for study, comparison, or inspiration are allowed in specific places only, with one constraint: **the framing must indicate reference / study / inspiration, never code transfer.**

Canonical operator-facing surface: the User Guide's "Credits and References" section. The single operator-facing place that names studied applications openly. Existing language is the reference: `"studied for ideas, ballistics, and protocol structure"`, `"No source code was copied"`, `"Lyra implements each idea natively in C++"`.

Allowed internal-surface mentions:

- docs/ comparison dossiers (docs/THETIS_VS_LYRA_*, design ledger, architectural mapping docs, **this file**) — for citations where the specific file path matters for the reader to follow the reference.
- Framing must use: `"similar to"`, `"as done in"`, `"studied from"`, `"see [file:line]"`, `"the architecture used in..."`.

Names KEPT OUT of:

- Shipped code
- Code comments
- Commit messages
- Operator-visible UI strings (status bar, dialogs, panels, About dialog, etc.)
- README

Forbidden framings — never use these, anywhere:

- "ported from"
- "copied from"
- "based on this code"
- "stolen from"
- Anything implying code lineage or that Lyra contains code from another project

The concern this rule prevents: any reader concluding `"Lyra contains code from [referenced app]"`. Lyra is original C++23/Qt6 code that mirrors the *architecture* of working open-source SDR applications — that is the message, in all surfaces.

Rule 3 — Operator-empirical authority When Rick's bench observation contradicts Claude's inference from code reading or documentation, Rick wins. Claude revisits the reading and finds where the model is wrong.

Rule 4 — Smallest revertable step → bench → next step Never two changes between benches. Each step ships in a state that is bench-validated before the next step starts. The rollback path remains intact at every commit.

Rule 5 — No patches; fix to reference behaviour Lyra's job is to be the reference's architecture, not to behave like the reference on top of a different architecture. Patching — changing a constant, adding an if-statement, wrapping a handler, or making any local symptom-suppressing change without first asking whether the architecture itself is wrong — is forbidden. See Rule 17 for the catch-myself protocol that enforces this.

Section B. TX Rip-and-Port — Phase Discipline

Rule 6 — Architecture-first, code-second Before any code lands for a new component, an architectural mapping document exists. It names every reference component, its Lyra replacement, the threading model, the queue topology, the state machine, and the call sites. Rick signs off on the mapping. Code follows the mapping. No code without prior mapping sign-off.

Rule 7 — Delete TX, don't refactor Current TX code moves to a tx-rip-archive branch and is deleted from main. New TX is built from empty files matching the architectural mapping. No code carry-over from the old TX implementation. The old code is preserved in git history for reference; it is not the foundation.

Rule 8 — Wire-inert until bench-validated New TX ships with HL2 wire output DISABLED. RX remains untouched throughout the TX rebuild — operator does not lose RX capability while TX is being ported. TX wire output activates only after the full chain passes bench validation against measurable reference behaviour.

Rule 9 — Bottom-up port, one reference component at a time Each component is read from the reference end-to-end first, then ported, then bench-validated, before the next component starts. No parallel multi-component work. No interleaving.

Rule 10 — PureSignal-shaped from day one The new TX architecture is designed with the sip1 tap point, calcc thread integration, and DDC0+DDC1 state-product routing as first-class concerns from the start — not bolted on later. PureSignal code lands in a later release, but the TX topology is built to receive it without rework.

Section C. Operating Practice

Rule 11 — Reference citation discipline For every component about to be ported, Claude READS the reference end-to-end FIRST. Code is then written Lyra-native (C++23 / Qt6 idiomatic — not a C# transliteration). If ANYTHING needs to vary from the reference's structure or behaviour — language idiom mismatch, threading primitive mismatch, any reason at all — Claude STOPS before any code is written, presents the variation with pros and cons, and discusses with Rick. Approved variations are recorded as declared deviations. Unauthorized variations are forbidden.

Rule 12 — Commit authorization Claude asks Rick before any substantive commit. **Substantive** = touches behaviour, architecture, file structure, or operator-facing surface. **Trivial** = typo in a comment Claude just wrote, build-only path fix, or formatting. When in doubt: ask.

Rule 13 — New file creation / green light Once Rick signs off on an architectural mapping document, that document is Claude's green light to create the files the document specifies. Per-file approval is not required INSIDE the approved scope. Files OUTSIDE the approved scope require Rick's approval.

Rule 14 — Bench-gate protocol

- Claude verifies what Claude can verify: clean build, log output matches expected reference behaviour, internal sanity checks, build-time and unit-test correctness.
- Anything requiring HL2-live-RF verification is Rick's: Claude ships in a state Rick can test, Rick benches on actual hardware, Rick reports yes / no working.

Rule 15 — "Done" definition

- **Self-verifiable component** (build + log + reference-matching measurable output) — Claude marks done.
- **HL2-on-air verification required** — Claude ships wire-inert or bench-staged. Rick verifies. Rick reports yes / no working. The component is "done" only after Rick's yes.

Rule 16 — Architectural decision review Claude's judgment call when to escalate mid-port. Rick trusts Claude to understand the end-state Lyra is being built toward: RX2-ready, PureSignal-ready, full-duplex-ready, reference-faithful TX path. Claude escalates when uncertain whether a choice fits the end-state.

Rule 17 — Catch-myself protocol When Claude notices itself about to change a constant, add an if-statement, or wrap a handler to make a symptom go away — Claude STOPS and asks itself:

"Why am I doing this when we have a working model to reference from?"

Then Claude goes back to the reference and finds the architectural fix, not the symptom-suppression. If the honest answer to the question is "because patching is easier here" — Claude tells Rick BEFORE writing anything, and they discuss.

Rule 18 — Rule conflict resolution When two rules tension each other (for example, Rule 1 reference-parity vs the practical reality that Qt is not C# and something must give), Claude STOPS, presents pros and cons, and Claude + Rick decide together. No silent picking-one-and-moving-on.

Section D. Documentation Discipline

Rule 19 — Design ledger Updated at the START of each new component port. Declares intent, reference citations, and any approved variation calls for the component.

Rule 20 — Architectural mapping doc and dossier Updated as each component completes. Records what shipped, what was bench-confirmed, and what was deferred.

Rule 21 — User guide Updated only when operator-facing behaviour changes.

Rule 22 — Reference notes (docs/refs/) Updated as Claude reads the reference, BEFORE code is written. Preserves the reading work even if Claude loses context or the session ends.

Section E. Amendments + Locked Additions

Rule 23 — Parity checkpoint (Amendment 1, locked 2026-06-05)

Each new section (one Phase-2+ component, or a natural sub-component when the parent is too large to checkpoint in a single pass) ships a written side-by-side comparison between the reference and Lyra BEFORE code lands. The checkpoint is a hard gate, not a suggestion.

Format. Three columns — **Aspect** / **Reference (file:line)** / **Lyra (proposed or shipped)**. One row per field, case, or behavior. Below the table, a single **VERDICT** line carrying one of three values:

- **■ PARITY** — byte-for-byte / behavior-for-behavior match.
- **■ ACCEPTABLE DEVIATION** — different in form, identical in effect (e.g. `std::mutex` vs `CRITICAL_SECTION`, `std::atomic` vs aligned `int`, C++ `enum class` vs C `enum`, `PascalCase` vs C-style names). Reason stated inline; operator awareness is implicit at sign-off.
- **■ OPERATOR-APPROVED DEVIATION** — substantive behavior change versus reference. Requires explicit operator sign-off recorded in the row BEFORE code lands. A **■** with no sign-off blocks the commit.

Where it lives.

- `docs/architecture/PARITY_CHECKPOINTS.md` — the tracked ledger. One section per component, commit-ordered. Operator-reviewable artifact that survives session compaction.

- **Commit message** — every commit that adds Phase-2+ behavior carries a one-line trailer: `Parity-checked: [■ / ■ / ■]`.

What counts as a section. One Phase-2 component (e.g. `RadioNet` field list = §1, `H12FrameComposer` 19-case dispatch = §2, `Ep2SendThread` `MMCSS` init + send loop = §3). Sub-sections allowed when natural. Each section ships ONCE; a later behavior change inside the same component lands as a delta-checkpoint amendment to the same section, not a re-issue.

Phase 1 exemption. Empty skeleton commits (default ctor/dtor only, populated in Phase 2 per the locked mapping doc) do not require a checkpoint — there is no behavior to compare yet. The Phase-2 commit that populates the skeleton carries the §N entry.

Cross-references. Rule 23 operationalizes Rule 1 (reference parity) and Rule 17 (catch-yourself protocol). The checkpoint is the artifact the catch-yourself reflex produces.

Rule 24 — Always verify against the reference (locked 2026-06-05)

Before any substantive assertion (parity-checkpoint row, byte-level claim, type claim, bit-position claim) AND before any commit, read the reference source verbatim and diff against the Lyra claim. No memory-based assertions. No agent-confidence-weighted assertions.

Why this rule exists. During §4a-prep on 2026-06-05 the source read revealed (a) `xmitBit` was claimed `volatile long` in signed §3.4 and shipped as `std::atomic` — actual reference type is plain `int`; (b) the §4a draft mapped case 1 = `RX1` — reference shows case 1 is the TX VFO; (c) case-0 byte composition in the draft was materially simpler than the verbatim source. All three were memory-based claims that an earlier audit (which read both source and Lyra) failed to catch because the audit was confidence-weighted, not source-line-weighted. Rule 24 locks the discipline that catches this class of error.

The discipline.

1. **For every parity-checkpoint row:** cite the reference by `file:line` AND open that line during drafting. Do NOT rely on memory of "the reference does X" — open the source and read it.
2. **For every type/bit-position/name claim:** copy the reference line verbatim into the checkpoint's "Reference" column. The Lyra "proposed" column then visibly diffs against it.
3. **Before every commit that touches Phase-2+ behavior:** re-read the reference sections the commit asserts to mirror, side-by-side with the Lyra code. Surface any discrepancy as a **■ DEFECT** in the commit's parity row, NOT in a follow-up.

4. **For prior signed checkpoints discovered to contain a memory-vs-source error:** amend the \$N checkpoint with a correction row + re-sign + fix the shipped code in the same commit (the §3.4 xmitBit correction on 2026-06-05 is the precedent).
5. **No exception for "small" claims:** a single-byte / single-bit wrong is still wrong. Rule 24 has no de minimis carve-out.

What this does NOT mean.

- It does not mean re-reading every reference line for every commit. It means re-reading the lines the commit asserts to mirror.
- It does not block memory-based COMMENTARY in chat / docs. It blocks memory-based CLAIMS that land in parity-checkpoint rows or shipped code.
- It does not require an agent to verify the work. The implementor reads the source directly; agents are optional second opinions.

Cross-references. Rule 24 strengthens Rule 1 (reference parity) and Rule 23 (parity-checkpoint methodology). Rule 17 (catch-yourself protocol) is the trigger; Rule 24 is the verification step that catch-yourself produces.

Rule 2 Amendment — Signed sign-off cells exempted (locked 2026-06-05)

Rule 2 (no reference-name leaks in tracked docs) does NOT apply to text inside signed operator sign-off cells in tracked architecture docs. The signed cell is a historical audit artifact ratified by operator signature at the time of signing; rewriting it to neutral language would alter the historical record.

Scope of the exemption.

- The exemption applies ONLY to text inside the table cells of sign-off rows (e.g. the "Sign-off log" tables in docs/TX_ARCHITECTURAL_MAPPING.md, the OPERATOR SIGN-OFF blocks in docs/architecture/PARITY_CHECKPOINTS.md).
- All surrounding doc body, headings, comments outside the signed cells remain subject to Rule 2 scrub.
- The exemption applies to ALREADY-SIGNED cells. NEW sign-off cells drafted post-2026-06-05 should be written in Rule-2-compliant language at the time of drafting — the exemption is for historical preservation, not for new drafting.

Why this exemption exists. During the 2026-06-05 post-§4b-1 audit, a single PowerSDR token was found in a 2026-06-04 Phase-0 sign-off cell at docs/TX_ARCHITECTURAL_MAPPING.md:1821. The cell records the operator's signed decision to skip a particular reference cross-reference path. Editing the cell post-signature would (a) rewrite the operator's ratified text and (b) erase the audit trail of WHICH decision was approved when. The exemption codifies "historical preservation > Rule 2 purity for signed cells."

Cross-references. Rule 2 (no reference-name leaks); Rule 23 (parity-checkpoint methodology — sign-off blocks are integral to checkpoints).

Section F. 2026-06-06 Session Amendments

These rules were added at the end of a long audit-driven session that exposed multiple PATCH-class deviations the prior rules (1-24) had not specifically caught. They tighten the catch-it-before-it-ships discipline.

Rule 25 — NO PORTING FROM PYTHON-LYRA FOR TX. EVER. (locked 2026-06-06) Python-Lyra TX was broken. It is NOT a valid source for lyra-cpp TX behavior. References to lyra-Python §15.xx patterns in CLAUDE.md are HISTORICAL CONTEXT only and may NOT be the basis for any code decision. If the only design source for an item is lyra-Python, that item is **STOP-AND-ASK** territory under Rules 11 + 18.

Rule 26 — ACCEPTABLE C↔C++23 IDIOM ALLOW-LIST (locked 2026-06-06) Operationalizes Rule 1 / Rule 23 ■ ACCEPTABLE DEVIATION classification. ONLY the following auto-acceptable as "C-IDIOM" without operator sign-off — anything else is a ■ PATCH:

- malloc/calloc → std::vector::assign or new T()
- memset(p,0,N) → std::fill_n or vector default
- CRITICAL_SECTION + EnterCriticalSection/LeaveCriticalSection → std::mutex + std::lock_guard
- CreateSemaphore + WaitForSingleObject + ReleaseSemaphore → std::counting_semaphore<1>
- WaitForMultipleObjects(2, ..., TRUE, INFINITE) (wait-all on max-count-1 semaphores) → std::condition_variable + paired-bool predicate
- HANDLE _beginthreadex → std::thread / std::jthread
- Sleep(N) → std::this_thread::sleep_for(Nms)
- unsigned char ptr[N] stack buffer → std::array stack

- UB signed-int byte aliasing → portable shifts via uint32 + signed cast
- C-style unscoped enum + shadow var → enum class + distinctly-named global
- C-struct → C++ class with embedded members (semantics preserved)
- C free function → C++ free function in namespace (no class wrapping unless operator-signed)
- File-scope global unsigned int x (non-atomic) → TU-scope std::uint32_t x or atomic; mark in comment which.

Anything outside this list is a PATCH per Rule 5, requires operator approval per Rules 11 + 18, and is recorded as a ■ DEFECT per Rule 23 until signed.

Rule 27 — 2-AUDIT GATE PER CHECKBOX (locked 2026-06-06) A checkbox in this file is marked complete ONLY after:

- **Audit #1** — Claude reads reference + Lyra side-by-side line-by-line, produces a CLEAN/PATCH table per Rule 23 methodology, file:line cited on BOTH sides per Rule 24.
- **Audit #2** — Independent re-verification. May be: (a) a fresh background agent, (b) operator bench-verification per Rule 14, (c) Claude re-reads in a fresh session with no prior context. Must produce its OWN file:line-cited table.
- BOTH audits must report ■ PARITY (or operator-signed ■/■ DEFERRED per Rules 11 + 23) for the item before the checkbox is marked.

Why this rule exists. The wire-layer audit round on 2026-06-06 caught 19+ PATCH-class items in the LIVE production TX path that earlier single-audit passes had silently classified as CLEAN. The 2-audit gate locks that pattern's prevention.

Cross-references. Rule 27 operationalizes Rule 23 (parity-checkpoint methodology) with a quantitative gate.

Rule 28 — BENCH ≠ REFERENCE-FAITHFUL (locked 2026-06-06) Amplifies Rule 15 ("Done" definition). RX-bench-passing alone is NECESSARY but NOT SUFFICIENT to mark a component done. Done requires BOTH:

- (a) Bench validation per Rule 14 / 15
- (b) Reference-parity Audit #1 + Audit #2 pass per Rule 27

Why this rule exists. "RX is solid" was claimed multiple times in pre-2026-06-06 sessions based on bench-passing alone. The 2026-06-06 LIVE-C audit found 9 PATCH + 8 MISSING items in the bench-passing RX path. Bench truth and reference truth are independent axes; both must be true.

Rule 29 — §3.9 OPERATOR-EMPIRICAL SIGN-OFF REGISTRY (locked 2026-06-06) Operationalizes Rule 3 (operator-empirical authority) with an explicit recorded sign-off discipline.

When Claude's reading of the reference and the operator's bench observation diverge AND the operator's bench observation is correct (per Rule 3, Rick wins) — the divergence does NOT silently propagate into shipped code. Instead:

1. The divergence is added to the **§3.9 Operator-Signed Divergences Registry** (section below in this file) with: divergence description, reference-source claim, bench-verified Lyra behavior with RTL/file:line provenance, and a **PENDING SIGN-OFF** flag.
2. Operator replaces **PENDING SIGN-OFF** with **SIGNED {YYYY-MM-DD}** when explicitly blessing the divergence.
3. Shipped code carries an inline // §3.9-N divergence per EXECUTION_PLAN.md – operator-signed {date} comment cite at the divergence site.
4. Stages that depend on the divergence are **BLOCKED** until sign-off.

Why this rule exists. Five bench-verified gateway-RTL divergences on the HL2+ ak4951v4 variant (20-bit seq mask, three telemetry slot re-maps, MoxEdgeFade SSB shim) were ambient in the live HL2Stream code without explicit operator sign-off. Stage 5/8 migrations would silently regress operator's bench-verified behavior unless these divergences are signed and intentionally ported forward.

Rule 30 — BUILD + RULE-2 GREP CLEAN PER COMMIT (locked 2026-06-06) Operationalizes Rule 2 + Rule 14. Every commit:

- cmd //c '._b.bat' exits 0 with zero error C[0-9]+ / fatal error.
- Rule 2 forbidden tokens (Thetis, thetis, PowersDR, powersdr, Console.cs, OpenHPSDR, openhpsdr) absent from shipped code (in src/, qml/); allowed in docs/ per Rule 2 internal-surface mention.
- Co-author trailer present: Co-Authored-By: Claude Opus 4.7 (1M context) .
- Backup bundle _backups/lyra- - .bundle if the commit is bench-gate-eligible per Rule 14.

- Commit message includes reference cites in body for Phase-2+ commits per Rule 24 step 3.
- Commit body carries Parity-checked: [■ / ■ / ■] trailer per Rule 23.

■ RULE-INDEX QUICK REFERENCE (cross-reference for code/commit cites)

Rule	One-line	Section
1	Reference-parity (mirror the reference)	A
2	Attribution discipline (no reference names in shipped code)	A
3	Operator-empirical authority (Rick wins on bench-vs-inference)	A
4	Smallest revertable step → bench → next step	A
5	No patches; fix to reference behaviour	A
6	Architecture-first, code-second (mapping doc before code)	B
7	Delete TX, don't refactor	B
8	Wire-inert until bench-validated	B
9	Bottom-up port, one reference component at a time	B
10	PureSignal-shaped from day one	B
11	Reference citation discipline (read reference end-to-end first)	C
12	Commit authorization (ask before substantive commits)	C
13	New file creation green-light (mapping doc unlocks scope)	C
14	Bench-gate protocol (Claude self-verifies; Rick HL2-verifies)	C
15	"Done" definition (self-verifiable vs HL2-on-air)	C
16	Architectural decision review (escalate end-state-relevant calls)	C
17	Catch-myself protocol (architectural fix, not patch)	C
18	Rule conflict resolution (STOP, pros/cons, joint decide)	C
19	Design ledger (start of each component port)	D
20	Architectural mapping doc + dossier (end of each component)	D
21	User guide (only when operator-facing behaviour changes)	D
22	Reference notes docs/refs/ (read-before-code, preserved)	D

Rule	One-line	Section
23	Parity checkpoint (Amendment 1: 3-col side-by-side gate)	E
24	Always verify against reference (source-line-weighted, not memory)	E
2-A	Signed sign-off cells exempted from Rule 2 scrub	E
25	NO porting from Python-Lyra for TX	F
26	C↔C++23 idiom allow-list (anything outside = PATCH)	F
27	2-audit gate per checkbox	F
28	Bench ≠ reference-faithful (both required for done)	F
29	§3.9 operator-empirical sign-off registry	F
30	Build + Rule-2 grep clean per commit	F

■ REFERENCE-PARITY VERIFICATION PROTOCOL (mandatory per item)

For EVERY non-trivial code change in Step 14 or Phase 3:

Pre-write

1. **Locate reference body.** Grep for the function name in the reference tree. Cite file:line. If not found → STOP-AND-ASK.
2. **First read.** Read the reference body in full. Note what it does in working memory.
3. **Second read.** Re-read the same body. Verify the first reading is accurate. Note any quirks/defects to preserve per Rule 24.
4. **Read Lyra side.** Read the current Lyra equivalent in full (if it exists).
5. **Build the parity table** before any code is written. Columns: Lyra (file:line) | Lyra quoted | Reference (file:line) | Reference quoted | Verdict | Notes. Verdicts per Rule 6.
6. **If the table shows anything not in the C-IDIOM allow-list (Rule 5) without operator sign-off → STOP-AND-ASK.**

Write

7. Write Lyra code that matches the reference behavior within Rule-5 idioms. Comments cite reference file:line.

Post-write

8. Build clean per Rule 9.
9. Rule 2 grep clean.
10. Run **Audit #1** — re-read both sides, re-build the parity table from scratch, confirm CLEAN.
11. Run **Audit #2** — independent re-verification (fresh agent OR fresh session OR operator bench).
12. Commit with descriptive message + co-author trailer + reference cites in body.
13. Mark the checkbox in this file with: [x] {date} {commit hash} audit1=PASS audit2=PASS.

Bench-gate stages (Stage 2 and Stage 8 of Step 14, plus any explicitly marked)

14. Operator bench-validates per the stage's bench-gate criteria.
15. Operator sign-off recorded inline next to the checkbox: OPERATOR-SIGNED {date}.

■ DAILY SESSION-START RITUAL

At the start of every session, Claude reads this file in this order:

- 1. **LOCKED OPERATING RULES** (Section above).
- 2. **§3.9 OPERATOR-SIGNED DIVERGENCES REGISTRY** (Section below).
- 3. **PROGRESS TRACKING — STEP 14** (current stage status).
- 4. **OPEN STOP-AND-ASK ITEMS** (if any).
- 5. **LAST-COMMIT LOG** (last 3-5 commits on tx-rebuild).

Then asks the operator: "What's the focus this session?" before proposing any code.

Sync the operator-readable copies (DOCX + PDF) each session-start

After any edit to this EXECUTION_PLAN.md, regenerate the operator-readable copies:

```
py -3.14 tools\sync_execution_plan.py --pdf
```

Produces:

- docs/EXECUTION_PLAN.docx — Word-readable mirror, regenerated from the MD source. Stamped with generation timestamp in the header.
- docs/EXECUTION_PLAN.pdf — paginated PDF mirror. Stamped with generation timestamp + page numbers in the footer.

EXECUTION_PLAN.md is the **source of truth**. The DOCX + PDF are auto-generated read-only views — never edit them directly. If they drift from the MD, re-run the sync script.

The sync script is committed at tools/sync_execution_plan.py. Requires Python with python-docx + reportlab (Python 3.14 on this machine has both).

■ OPEN STOP-AND-ASK ITEMS (Claude blocks here)

(none currently — populate as they arise)

■ §3.9 OPERATOR-SIGNED DIVERGENCES REGISTRY

Per Rule 29 — bench-verified gateway-RTL divergences that explicitly deviate from reference source. Each entry needs operator sign-off date OR is REJECTED → marked for REVERT TO REFERENCE.

OPERATOR RULING 2026-06-06 — ALL FIVE BELOW REJECTED FOR SIGN-OFF. None match the locked reference (Thetis 2.10.3.13 + networkproto1.c). Per Rule 1 (DO AS THE REFERENCE DOES) and Rule 5 (NO PATCHES), they are reclassified as ■ PATCH and queued for REVERT TO REFERENCE during the Step 14 stage work. The §3.9 sign-off mechanism remains in force for **future** genuine bench-vs-reference disagreements the operator chooses to bless — but these 5 are not that.

If a revert produces wrong telemetry / TX click on N8SDR's HL2+ bench, that is a Rule 18 STOP-AND-ASK ("reference says X, bench says Y — operator decides") — NOT a Claude-decided divergence shipped under §3.9 cover.

#	Divergence	Reference says (LOCKED — do this)	Lyra current (PATCH — revert)	Status
§3.9-1	EP6 seq counter width	32-bit per networkproto1.c:191-194	20-bit mask kSeqMask20 = 0x000FFFFF in hl2_stream.cpp:1091-1118	■ REVERTED 2026-06-07 — commit 3b7888b (Task #115)
§3.9-2	Telemetry slot semantics — case 0x08 C1:C2	exciter_power per networkproto1.c:506-507	decoded as temp on HL2+ ak4951v4	■ REVERTED 2026-06-07 — commit 42f66c2 (Task #115)
§3.9-3	Telemetry slot semantics — case 0x10 C3:C4	user_adc0 ("AIN3 MKII PA Volts")	decoded as PA current	■ REVERTED 2026-06-07 — commit 42f66c2 (Task #115)

#	Divergence	Reference says (LOCKED — do this)	Lyra current (PATCH — revert)	Status
§3.9-4	Telemetry slot semantics — case 0x18	user_adc1 (C1:C2) + supply_volts (C3:C4) per networkproto1.c:516-517	treated as dead/junk	■ REVERTED 2026-06-07 — commit 42f66c2 (Task #115)
§3.9-5	MoxEdgeFade for SSB	WDSP TXAUSlewCheck at wdsp/TXA.c:819-824 returns 0 for SSB → no envelope shaping for SSB	Lyra-native cos ² shim in src/mox_edge_fade.cpp (336 LOC + mid-fade reversal math)	■ REVERTED 2026-06-07 — commit 12e7acc (Task #115). 336 LOC DELETED + FSM rewire + Settings UI removed. Operator confirmed amp-safety call: rfDelayMs_ (50 ms default) is now sole hot-switch protection, matching reference.

All 5 reverts shipped Step 14 Stage 1.5 (pre-Stage-2 pass). The locations originally tagged Stage 5 / Stage 8 below were preemptive — the LIVE production hl2_stream.cpp carried the patches directly, and reverting in-place restores reference parity for the active code path. The clean src/wire/Ep6RecvThread.cpp was already reference-faithful (no §3.9-1 mask, reference-verbatim slot decode) and required no edits.

Audit gate status per Rule 27 (2-audit gate):

#	Audit #1 (Claude side-by-side parity)	Audit #2 (operator bench-verify on HL2+)
§3.9-1	■ PASS (commit 3b7888b — file:line cited on both trees in commit message)	■ PENDING (low priority) — operator next-bench run; verify EP6 seq error counter behaves sensibly (may tick once every ~3:27 min on the ak4951v4 20-bit gateway wrap — that is now the correctly-flagged event, not a false alarm). Bench 1 (2026-06-07) showed RX working cleanly, no continuous seq-error flood — implicit pass; explicit counter inspection deferred.
§3.9-2/3	■ PASS (commits 42f66c2 slot decode + 6d0e476 HL2 reinterpretation per console.cs:24937-24941)	■ SIGNED 2026-06-07 — Operator bench Round 2 on 6d0e476: banner T reads real ambient board temp; PA reads 0.00 A idle (correct — no TX/PA bias engaged at rest); on dead-key attempt, brief ~0.2 A draw observed before crash — matches reference (Thetis) bias-engagement signature exactly. Reference parity confirmed.
§3.9-4	■ PASS (commit 42f66c2; slot 0x18 decode removed in 6d0e476 per gateway Verilog _hl2src/hl2_rtl_control.v:475 "Unused in HL")	■ SIGNED 2026-06-07 — Operator bench Round 2 on 6d0e476: V field correctly shows n/a / 0.00 V (matches reference — Thetis doesn't display supply on HL2 either; console.cs:26758-26761 reuses the Volts slot for °C). The previously-displayed 12.3 V was a Lyra-side bit-pattern coincidence, NOT a real measurement.

#	Audit #1 (Claude side-by-side parity)	Audit #2 (operator bench-verify on HL2+)
§3.9-5	■ PASS (commit 12e7acc)	■ DEFERRED — SSB TX acoustic verification requires TX wire-up (Stages 7-10 of STEP14_PLAN.md). TX subsystem is intentionally non-functional during the wire-layer rebuild (per <code>main.cpp:549-559</code>); clicking MOX/TUN today crashes Lyra by design, not from this revert. Bench Round 2 observed brief PA bias engagement (~0.2 A momentary draw) on dead-key attempt before crash — confirms PA-enable wire bit + bias engagement work correctly; the crash is downstream in the TX DSP path that's intentionally torn out for rebuild.

Per Rule 27 the §3.9-N checkboxes are NOT considered "complete" until both audits pass. Any FAIL on Audit #2 is a Rule 18 STOP-AND-ASK — surface to operator before any in-place fix. This file gets updated with "■ SIGNED {YYYY-MM-DD}" rows when the operator bench-confirms each item.

On the §15.26 "supply at 0x00 C1:C2 >>4" claim: that was disproven during this Audit #2 cycle via direct read of the operator's gateway Verilog (`_hl2src/hl2_rtl_control.v:472`). Slot 0x00 C1:C2 carries `dsiq_status + VERSION_MAJOR`, NOT supply. The 12.3 V Lyra previously displayed was a coincidence of bit patterns scaled through a wrong formula — not a real measurement. Reference (Thetis) does not display supply voltage on HL2; the status-bar "Volts" label slot is reused to show temperature with a "C" suffix per `console.cs:26758-26761`. Lyra now matches this. Future supply telemetry, if wanted, must come from a separate I2C2 readback (not EP6, not §3.9 — its own work item).

#	Where the revert landed	Reference cite
§3.9-1	<code>hl2_stream.cpp</code> — <code>expectedSeq = seq + 1</code> (no mask)	<code>networkproto1.c:191-194</code> MetisReadDirect
§3.9-2 / §3.9-3 / §3.9-4	<code>hl2_stream.cpp</code> slot decode + <code>hl2_stream.h</code> <code>atomics</code> ; <code>hl2TempC()</code> returns NaN (HL2 board temp is I2C2, separate work item)	<code>networkproto1.c:498-525</code> HL2 read loop
§3.9-5	<code>src/mox_edge_fade.{cpp,h}</code> DELETED; <code>hl2_stream.cpp</code> FSM <code>fsmKeyupTxOff()</code> reference-faithful keyup; EP2 packer emits raw TX I/Q	<code>wdsp/TXA.c:819-824</code> <code>TXAUslewCheck</code> returns 0 for SSB

Process going forward: if a 6th genuine bench-vs-reference disagreement surfaces (something the operator WANTS to override the reference on, not just a Claude-invented "smart" divergence), add a new §3.9-N row with **PENDING SIGN-OFF**. Sign-off requires the operator to explicitly bless it — same discipline as today, just with the bar that Rule 1 + Rule 5 reject the default. The five rejected above are NOT examples of the mechanism succeeding; they are examples of the mechanism filtering out PATCHES that should never have shipped silently in the live HL2Stream code in the first place.

■ PROGRESS TRACKING — STEP 14 (Wire-Layer Migration)

Reference: `docs/architecture/STEP14_PLAN.md` (Stages 1-10 spec).

Stage 1 — Wire-Layer Singleton + Bind, Wire-INERT

■ SHIPPED earlier 2026-06-06 (compile-and-link only; RX bench passed)

- Audit #1: PASS (today's wire-layer round)
- Audit #2: PASS (operator RX bench-verified)
- **Status:** ■ COMPLETE

Stage 1.5 — Wire-Layer Reference-Parity Fix Pass (audit cleanup, today's work)

- Commit A — drop FIX 8 hwriteThreadInitSem vestige (Ep2SendThread.cpp)
 - Commit B — Ep6RecvThread Round-2 carryovers (MetisReadDirect local-stack 1074-byte buffer + rcvpktp1 lock + reference-verbatim seq check + drop g_seq_seen + drop WSACloseEvent on stop + drop duplicate metis_wire_bind)
 - Commit C — cleanup deletes (drop AvRevertMmThreadCharacteristics both threads + drop ForceCandC defensive null guards + drop write_main_loop_hl2 defensive asserts + drop scheduler prologue C1..C4 re-zero + drop 19 per-case assert(prn != nullptr) defensive guards + simplify setters to pure single-line writes + add bandwidth_monitor_reset() stub anchor)
 - Commit D — OutboundRing reference-parity (fix initial state lr/iq_consumed=false; producer order memcpy → set_ready → notify → wait; drop outbound_unblock + outbound_stop; bounded wait_for(100ms) poll)
 - Commit E — MetisFrame 2-layer MetisWriteFrame → send_packet restored (per-call sockaddr from g_mmetis_addr_be + prn->base_outbound_port; prn->sndpkt lock around sendto; bandwidth_monitor_out anchor; sendto failed error log)
 - Commit F (cosmetic) — mb shadow warning fix in hl2_stream.cpp::composeCC case-10
- Audit #1: PASS (4-agent wire-layer audit confirmed 196 CLEAN comparisons + all PATCH-class findings resolved)
 - Audit #2: PENDING (operator bench-verify — wire-layer tree still INERT, so bench-verify happens via Stage 2 first-wire test)
 - **Status:** ■ COMPLETE (pending Audit #2 via Stage 2 bench-gate)
-

Stage 2 — ForceCandC::prime(3, tx_freq, rx_freq) Insert, Wire-LIVE on First HL2 Talk

Bench-critical commit. First time new wire layer talks to real HL2 hardware.

Reference order (verbatim per networkproto1.c:427-434 + netInterface.c:50, 60-66):

```
HL2Stream::open():
  1. UDP socket open + state clear (existing)
  2. prn = create_rnet(); metis_wire_bind(); outbound_init(); ← Stage 1 (done)
  3. SendStartToMetis equivalent — buildControlPacket(true) + sendto on socket_
    (hoisted from old txWorkerLoop; mirrors netInterface.c:50)
  4. Ep6RecvThread::start(socket_) ■■■■ caller blocks on init-sem ■■■■
                                     ■
In Ep6RecvThread::run_loop():
  a. allocate FPGA buffers (calloc(1024)) [networkproto1.c:427] ■
  b. ForceCandCFrame(3) [networkproto1.c:430] ■
  c. WSACreateEvent [networkproto1.c:433] ■
  d. WSAEventSelect(socket, event, FD_READ) [networkproto1.c:434] ■
  e. init_sem.release() [netInterface.c:61-63]■
  f. while (io_keep_running) { WSAWaitForMultipleEvents + dispatch }
  5. caller unblocks; seed `txSeq_` from `metis_out_seq_num()`; spawn TX worker
```

STOP-AND-ASK first: the plan's "stage-boundary call" — does Stage 2 spawn Ep6RecvThread as the LIVE rcv path (collapsing Stage 3+6 into Stage 2), OR does it leave the old rxworker_jthread live and Ep6 only does priming+park-until-Stage-6? Operator must answer before Stage 2 starts.

Per-item checklist:

- **Stage-boundary call decision** (operator answers question above)
- Pre-write: read networkproto1.c:427-434 twice; verify against today's Ep6RecvThread::run_loop() body
- Pre-write: read netInterface.c:50, 60-66 twice; build parity table for the SendStartToMetis hoist
- Add init-sem release point in Ep6RecvThread::run_loop AFTER WSAEventSelect AND force_candc_frame(3) complete
- Add init-sem acquire in HL2Stream::open() after Ep6RecvThread::start() returns
- Hoist buildControlPacket(true) + ::sendto from txWorkerLoop:2451-2461 into HL2Stream::open() BEFORE the Ep6 spawn
- Delete the open-time START send from txWorkerLoop
- Seed txSeq_ = lyra::wire::metis_out_seq_num() AFTER init-sem releases, BEFORE txworker_ spawns
- Build clean per Rule 9
- Rule 2 grep clean

■ Audit #1: []

■ Audit #2: []

■ Operator bench-gate per STEP14_PLAN.md §Stage 2 (cold open → 6 priming datagrams emit → RX still works on known signal → no abnormal seq/framing errors over 5 min)

■ OPERATOR-SIGNED {date}

• **Status:** ■ IN PROGRESS (sub-staged 2a + 2b — see below)

Stage 2 sub-stage decisions (operator-locked 2026-06-07)

Stage-boundary call (the OLD pre-Stage-2 STOP-AND-ASK): Option B chosen — Stage 2 retires the old rxworker_jthread; the new Ep6RecvThread becomes the LIVE EP6 recv path on this commit. The old txworker_jthread stays alive (TX retires at Stage 8).

Init-sem release position (caught by Read-Twice 2026-06-07): the existing Stage 2 checklist item "Add init-sem release point in Ep6RecvThread::run_loop AFTER WSAEventSelect AND force_candc_frame(3) complete" was a Lyra-native divergence from reference. Reference releases hReadThreadInitSem in MetisReadThreadMain at networkproto1.c:249 — AFTER MMCSS classification, BEFORE calling MetisReadThreadMainLoop_HL2() (which is where priming + WSAEventSelect happen). Operator directive 2026-06-07: **"Make like reference always — PureSignal requires it"** (single shared outbound seq counter across priming + steady-state).

Telemetry storage: HL2Stream accessors (hl2TempC / paCurrentA / fwdPowerW / revPowerW / hl2SupplyV) read prn->... direct — sub-decision (a). The Lyra-native telemetry atomic mirrors (telExciterRaw_, telUserAdc0Raw_, telFwdRaw_, telRevRaw_) retire in Stage 2b.

Single-commit Stage 2 expanded to 2a + 2b (operator-approved 2026-06-07) because Read-Twice surfaced ~7 operator-facing instruments in rxWorkerLoop that need re-homing in addition to the protocol-level migration. Sub-staging contains the build-break risk; each sub-stage individually bench-testable + revertable. The end-state remains 100% reference-faithful.

Stage 2a — Ep6RecvThread reference-position prep, wire-INERT (2026-06-07)

Forward-compat additions to the wire layer. NO production behavior change — Ep6RecvThread is still not instantiated/started anywhere in src/hl2_stream.cpp (rxWorker_ remains the live RX path). Sets up the surfaces Stage 2b will wire LIVE.

Per-item checklist:

■ Pre-write: read networkproto1.c:240-261 (MetisReadThreadMain) + :422-586 (MetisReadThreadMainLoop_HL2) twice — confirmed init-sem release at :249 BEFORE the loop call at :253

■ Pre-write: read Ep6RecvThread.{h,cpp} + ForceCandC.{h,cpp} + MetisFrame.{h,cpp} + RadioNet.h twice

■ **Init-sem release moved to reference position** in Ep6RecvThread::run_loop — released immediately after MMCSS Pro Audio classification, BEFORE the per-thread init block (FPGA buffer alloc, priming pass, WSAEventSelect). Mirror of reference at networkproto1.c:249 + netInterface.c:60-66 handshake semantics.

■ **Misleading comment block fixed** — the prior "Reference: implicit via ReleaseSemaphore ... typically after the priming pass" docstring was factually incorrect; replaced with accurate cite of reference position + PureSignal rationale (single shared MetisOutBoundSeqNum).

■ **HW PTT-in sink scaffolding added** — new Ep6HwPttSink = std::function typedef + set_hw_ptt_sink(sink) method on Ep6RecvThread. Fired from inside decode_status_header immediately after the prn->ptt_in = cc[0] & 0x01 shadow write (mirrors reference write site at networkproto1.c:496). Wire-INERT until Stage 2b registers a consumer from HL2Stream::open().

■ Build clean per Rule 9 — PENDING operator

■ Rule 2 grep clean — verified zero matches on Thetis/thetis/PowerSDR/powersdr/Console.cs/OpenHPSDR/openhpsdr across the edited files (one violation caught + corrected mid-edit, no remaining matches)

■ Audit #1: ✓ (cite confirmation in this section)

■ Audit #2 PENDING — operator bench: launch Lyra, open stream, confirm RX works identically (no behavior change expected since new layer is still wire-INERT in production paths). Expected delta vs HEAD: zero.

■ OPERATOR-SIGNED {date}

• **Status:** ■ EDITS COMPLETE, awaiting build + Audit #2

Stage 2a — Audit #1 (Claude side-by-side parity)

Element	Lyra-side	Reference	Status
Init-sem release point	Ep6RecvThread.cpp:run_loop — released immediately after MMCSS Pro Audio classification, BEFORE FPGA buffer alloc / priming / WSAEventSelect	networkproto1.c:249 ReleaseSemaphore(prn->hReadThreadInitSem, 1, NULL); — BEFORE MetisReadThreadMainLoop_HL2() call at :253	✓ Position matches
Spawn-side handshake	Ep6RecvThread::start(socket_) — prn->hReadThreadInitSem.acquire() blocks caller until thread releases (no behavior change in this commit since start() is not yet called)	netInterface.c:60-66 CreateSemaphore + _beginthreadex + WaitForSingleObject pattern	✓ Idiom translation matches
HW PTT-in shadow write	Ep6RecvThread.cpp::decode_status_header — p->ptt_in = static_cast(cc[0] & 0x01); (unchanged from §3.9 era)	networkproto1.c:496 prn->ptt_in = ControlBytesIn[0] & 0x1;	✓ Byte-identical
HW PTT-in sink dispatch	Ep6RecvThread.cpp::decode_status_header — hw_ptt_sink_(static_cast(cc[0] & 0x01)) fires on every non-I2C status decode	Reference has no equivalent sink (consumer reads prn->ptt_in directly elsewhere); C↔C++ idiom translation for the Lyra Q_OBJECT consumer boundary	✓ Acceptable Lyra-native C++ idiom
Rule 2 grep	Zero matches on src/wire/Ep6RecvThread.{h,cpp}	—	✓
Wire bytes vs HEAD	Identical — no production path calls Ep6RecvThread::start() yet (only hl2_stream.cpp lines 723-728 spawn workers, and rxWorker_/txWorker_ are unchanged)	—	✓ Zero behavior delta

Stage 2b — Wire-LIVE EP6 migration + txWorker_ shared-counter (NEXT)

Wire-LIVE commit. Retires rxworker_ jthread + rxworkerLoop body; migrates txworker_ send path to the shared g_metis_out_seq_num via metis_write_frame() (the load-bearing PureSignal posture). Re-homes all 7 operator-facing instruments that currently live inside rxWorkerLoop.

Locked architectural invariants (carried from Stage 2 plan):

- New Ep6RecvThread is the LIVE EP6 recv path (Option B)
- Ep6RecvThread::start() returns when init-sem releases (already at reference position post-Stage-2a). Priming + WSAEventSelect happen async on EP6 thread.
- START packet hoisted from txworkerLoop to open() body, BEFORE ep6Thread_.start() (matches netInterface.c:50)
- txWorker_::sendDatagram migrated to lyra::wire::metis_write_frame(0x02, payload_1024) — consumes shared g_metis_out_seq_num. Local txSeq_ deleted.
- 7 instruments re-homed (see Stage 2b planning notes below)

Per-item checklist (planning, NOT YET STARTED):

- Pre-write: complete read of rxworkerLoop body (lines 1013-1410 area) end-to-end (started in Stage 2a Read-Twice; ~300 lines remaining)
- Pre-write: read full txworkerLoop::sendDatagram lambda + buildEp2KeepaliveTemplate body twice
- Pre-write: design side-tap routing for RX1 dBFS RMS + mic dBFS RMS instruments (whether to put them in WDSP engine / Hl2Ep6MicSource / second Router sink — operator decision needed)
- Plan + 2-agent red-team check before code (Stage-2b-class change merits the same discipline as Stage 2 itself)
- HL2Stream.h: add Ep6RecvThread ep6Thread_ + ep6Thread() getter; delete rxworker_, rxworkerLoop() decl, iqSink_/setIqSink, micConsumer_/mtx/setMicConsumer, txSeq_, telemetry raw atomics (telExciterRaw_/telUserAdc0Raw_/telFwdRaw_/telRevRaw_)

- HL2Stream::open: hoist START send, wire sinks on ep6Thread_, ep6Thread_.start(socket_), delete rxWorker_ spawn, delete txSeq_.store(0)
- HL2Stream::close: replace rxWorker_.stop/join with ep6Thread_.stop()
- HL2Stream::txWorkerLoop: delete open-time START send; migrate sendDataGram to metis_write_frame(); delete txSeq_.fetch_add + pktBytes[4..7] seq patching
- HL2Stream::rxWorkerLoop: DELETE entire function body (~380 lines)
- HL2Stream accessors: hl2TempC/paCurrentA/fwdPowerW/revPowerW switch to read prn->... direct
- HL2Stream::Auto-LNA: switch adcOverloadNow_ consumer to read prn->adc[0].adc_overload direct
- HL2Stream: wire ep6Thread_.set_hw_ptt_sink([this](bool on){ /* edge detect + opt-in gate + invokeMethod */ })
- RX1 dBFS RMS: design + implement re-homing (TBD)
- Mic dBFS RMS: design + implement re-homing (TBD)
- Seq/framing/total dg counters: add accessors to Ep6RecvThread, route HL2Stream Q_PROPERTY getters
- main.cpp:252: setIqSink → lyra::wire::register_sink(...)
- mic_source.cpp:15: setMicConsumer → ep6Thread().set_mic_sink(...)
- Build clean
- Rule 2 grep clean
- Audit #1: side-by-side parity table for all 7 instrument migrations
- Audit #2: operator bench — cold open, RX1 audio on known signal, panadapter, all telemetry banner fields, Auto-LNA back-off, foot-switch (if hwPttEnabled), 30-min soak, ×5 stop/restart, MOX bit flip without PA
- OPERATOR-SIGNED {date}
- **Status:** ■ AWAITING STAGE 2A SIGN-OFF + 2B PLANNING

Stage 3 — Stand Up Ep6RecvThread In Parallel, Mute Sinks, Read-Only Compare

(Folds-in only if Stage 2 did NOT collapse this in.)

Per-item checklist:

- Pre-write: re-read Ep6RecvThread::process_datagram body twice
- Expose Ep6RecvThread::process_datagram() publicly
- Call from rxWorkerLoop AFTER existing validation, BEFORE old iqSink_ dispatch
- **REVERT §3.9-1 (rejected):** Ep6RecvThread reads the full 32-bit seq per networkproto1.c:191-194 — do NOT port the 20-bit mask. If bench shows wraparound issues on N8SDR's HL2+, STOP-AND-ASK per Rule 18.
- Verify Ep6's start() machinery stays dormant in this stage (no second recvfrom on shared socket)
- Build + Rule 2 + Audit #1 + Audit #2
- Operator bench-gate: A/B comparator — seq error rates, per-DDC IQ, mic harvest, telemetry all match
- OPERATOR-SIGNED {date}
- **Status:** ■ NOT STARTED

Stage 4 — Migrate First Consumer (WDSP IQ Sink) Off iqSink_ Onto Router::register_sink()

Per-item checklist:

- Pre-write: re-read Router + xrouter + twist reference bodies twice
- Register WDSP RX1 IQ sink via Router::register_sink() on Ep6RecvThread
- HL2Stream's iqSink_ dispatch made conditional (if (!routerActive_))

- Build + Rule 2 + Audit #1 + Audit #2
 - Operator bench-gate: RX1 audio A/B vs HEAD
 - OPERATOR-SIGNED {date}
 - **Status:** ■ NOT STARTED
-

Stage 5 — Migrate Mic + Telemetry + PTT-In Consumers Off HL2Stream Onto Ep6RecvThread sinks

Per-item checklist:

- Pre-write: re-read networkproto1.c:478-525 (telemetry decode) twice
 - **REVERT §3.9-2 / §3.9-3 / §3.9-4 (rejected):** Ep6RecvThread telemetry decode follows networkproto1.c:498-518 (HL2 read loop) slot map verbatim — case 0x08 C1:C2 = exciter_power, case 0x10 C3:C4 = user_adc0, case 0x18 C1:C2 = user_adc1 + C3:C4 = supply_volts. Do NOT carry the HL2+-bench-tuned re-map. If bench shows wrong telemetry values on N8SDR's HL2+, STOP-AND-ASK per Rule 18.
 - **AUDIT FOLD (real ungated):**
 - Verify micDecimationCount reset at Ep6RecvThread run_loop entry (today's Commit B should have this)
 - Verify seqErrors reset at run_loop entry
 - Verify first-frame seq check is reference-verbatim (today's Commit B)
 - Verify ptt_in/dash_in/dot_in shadows preserved per Rule 24 (reference defects intact)
 - Add case 0x20 per-ADC overload decode (currently missing in Ep6RecvThread::decode_status_header)
 - **AUDIT FOLD HW-PTT forwarder:** Lyra-native opt-in routed via Ep6 sink (preserve §15.26 RESOLVED-CORRECTION hwPttEnabled_gate, default OFF)
 - Mic source consumer routed via Ep6RecvThread::set_mic_sink
 - Telemetry consumer routed via Ep6RecvThread::set_telemetry_sink
 - Build + Rule 2 + Audit #1 + Audit #2
 - Operator bench-gate: PA-current readout matches §15.26 last-known-good (1.76A at full tune); foot-switch (if opted in); mic feeds DSP correctly
 - OPERATOR-SIGNED {date}
 - **Status:** ■ NOT STARTED
-

Stage 6 — Activate Ep6RecvThread::start() As Its Own std::thread, Delete HL2Stream::rxWorkerLoop

Per-item checklist:

- Pre-write: re-read MetisReadThreadMainLoop_HL2 start-to-end twice
 - Ep6RecvThread::run_loop becomes the live recv path
 - Delete HL2Stream::rxWorkerLoop body
 - Delete Stage 3's inline tap from old rxWorkerLoop (now dead code)
 - Build + Rule 2 + Audit #1 + Audit #2
 - Operator bench-gate: 30-min soak + 5× stop+restart cycles, no regressions
 - OPERATOR-SIGNED {date}
 - **Status:** ■ NOT STARTED
-

Stage 7 — Migrate TX-Side Setters Onto FrameComposer Free Functions

Per-item checklist:

- Pre-write: re-read writeMainLoop_HL2:869-1201 setter call sites twice

- HL2Stream setters (setRx1FreqHz, setTxFreqHz, setLnaGainDb, setTxStepAttnDb, setPaOn, setMicBoost, etc.) dual-write: existing atomic AND lyra::wire::set_* free function
- **AUDIT FOLD:** HL2Stream::set_pa_on → set ApolloTuner (0x08) + ApolloFilt (0x04) globals correctly so FrameComposer case-10 emits the §15.26 PA-on C2 = 0x4C
- **AUDIT FOLD:** HL2Stream::setLnaGainDb write path matches FrameComposer case-11 RX-branch wire byte (0x40 | ((g+12) & 0x3F))
- **AUDIT FOLD:** HL2Stream::setTxStepAttnDb write path matches FrameComposer setter (31 - signed_db)
- Build + Rule 2 + Audit #1 + Audit #2
- No bench gate (inert debug read-back only). Verify FrameComposer reads match HL2Stream atomics via diagnostic dump.
- OPERATOR-SIGNED {date}
- **Status:** ■ NOT STARTED

Stage 8 — Migrate EP2 Egress Onto Ep2SendThread + write_main_loop_hl2(). SECOND HL2 BENCH GATE.

Bench-critical commit. Load-bearing TX-side migration. Resolves all 19 LIVE-A composeCC PATCHes + all 19 LIVE-B txWorkerLoop PATCHes in one cut.

Per-item checklist:

- Pre-write: re-read sendProtocol1Samples:1204-1267 twice
- Pre-write: re-read MetisWriteFrame:216-237 + sendPacket@network.c:1382-1402 twice
- HL2Stream::open spawns Ep2SendThread instead of txWorker_ jthread
- LRIQ samples flow: HL2Stream audio path → outbound_push_lr/iq → Ep2SendThread → MetisFrame
- **REVERT §3.9-5 (rejected):** DELETE src/mox_edge_fade.cpp + its caller. SSB MoxEdge path becomes a no-op per WDSP TXAuslewCheck semantics (wdsp/TXA.c:819-824 returns 0 for SSB → no envelope shaping). Reference doesn't envelope-shape SSB at all; Lyra mustn't either. If bench shows SSB keydown/keyup clicks on N8SDR's HL2+, STOP-AND-ASK per Rule 18.
- **AUDIT FOLD §15.20 host TX-timeout:** verify lyra-cpp equivalent exists; add if missing
- Delete HL2Stream::txWorkerLoop body
- Delete HL2Stream::composeCC body (replaced by FrameComposer::write_main_loop_hl2)
- Build + Rule 2 + Audit #1 + Audit #2
- **Operator bench-gate:** first-RF + PA-current readout MUST match §15.26 last-known-good (~1.76A at full tune, ~5W on Palstar). 30-min soak. 5× stop+restart cycles. No regressions.
- OPERATOR-SIGNED {date}
- **Status:** ■ NOT STARTED

Stage 9 — Delete Stage-7 Dual-Write + Retire HL2Stream-side TX Atomics + Unused State

Per-item checklist:

- Pre-write: enumerate all HL2Stream TX atomics that became dead after Stage 8
- Delete dual-write paths
- Delete dead atomics
- Delete HL2Stream::queueTxAudio, txIqSource_, etc. per STEP14_PLAN.md
- Build + Rule 2 + Audit #1 + Audit #2
- Operator bench-gate: functional regression check (key, hear sidetone if any, telemetry, RX continues)
- OPERATOR-SIGNED {date}
- **Status:** ■ NOT STARTED

Stage 10 — Final Cleanup + Doc Sweep

Per-item checklist:

- Delete unused `destStorage_` member in `HL2Stream.h`
 - Drop the `hwriteThreadInitSem` member from `RadioNet.h` IF strict reference parity requires (reference DECLARES it but never uses; can stay for parity)
 - Update `CLAUDE.md` to reflect Step 14 complete + Phase 3 next
 - Update `docs/architecture/PARITY_CHECKPOINTS.md`
 - Sweep Rule 2 forbidden tokens project-wide (Task #73)
 - Final build + Rule 2
 - No bench gate (doc + cleanup only)
 - **Status:** ■ NOT STARTED
-

■ PROGRESS TRACKING — PHASE 3 (after Step 14)

Phase 3 begins ONLY after Step 14 Stage 10 is OPERATOR-SIGNED.

Phase 3.1 — `src/tx/TrSequencing.h` *populate fields*

- Pre-write: re-read `CLAUDE.md` §15.26 LOCKED reconcile + Thetis DB export values
- Add fields: `rf_delay_ms`, `mox_delay_ms`, `space_mox_delay_ms`, `ptt_out_delay_ms`, `key_up_delay_ms`
- Defaults: `rf_delay=50` (operator-tunable 1..75 hot-switch range), `mox_delay=10`, `space_mox_delay=13`, `ptt_out_delay=20`, `key_up_delay=10`
- Build + Audit #1 + Audit #2
- **Status:** ■ NOT STARTED

Phase 3.2 — `src/tx/KeydownSequence.cpp` *implement per §15.25 ground truth*

- Pre-write: re-read `console.cs::chkMOX_CheckedChanged2:30058+` twice
- Implement: RX-DSP stop (`SetChannelState(id(0,0),0,1) dmode=1 BLOCKING flush`) → ATT-on-TX → UpdateDDCs equivalent → MOX-bit set → if !CW: `rf_delay` → TX-DSP start (`SetChannelState(id(1,0),1,0)`)
- Build + Audit #1 + Audit #2
- Bench-gate: dummy-load TX, scope keydown order matches reference
- **Status:** ■ NOT STARTED

Phase 3.3 — `src/tx/KeyupSequence.cpp` *implement per §15.25 ground truth*

- Pre-write: re-read `console.cs::chkMOX_CheckedChanged2:30350+` twice
- Implement: TX-DSP off (`SetChannelState(id(1,0),0,1) dmode=1 BLOCKING TX downslew flush`) → `mox_delay` sleep → clear MOX bit → `ptt_out_delay` sleep → RX-DSP restart
- Build + Audit #1 + Audit #2
- Bench-gate: dummy-load TX, scope keyup order matches reference (no key-click/splatter)
- **Status:** ■ NOT STARTED

Phase 3.4 — `src/tx/AttOnTxPolicy.cpp` *implement*

- Pre-write: re-read `console.cs:30293-30327` (keydown) + `:30391-30410` (keyup) twice
- Implement: keydown save per-band RX1 step-att + force-31 (when PS-A off or CW); keyup restore
- Build + Audit #1 + Audit #2

■ Bench-gate: keying does NOT overload RX ADC (panadapter doesn't go wide on TX)

■ **Status:** ■ NOT STARTED

Phase 3.5 — `src/tx/PttFsm.cpp` *implement body*

■ Pre-write: re-read CLAUDE.md §15.25 FSM ground truth (PttSource set + resolver pattern)

■ Implement: PttSource enum, `_resolve()` precedence `MOX_TX > CW_TX > VOX_TX > TUN_TX > RX`, source-set funnel

■ Build + Audit #1 + Audit #2

■ Bench-gate: SW-MOX + HW-PTT share state correctly; no phantom keying

■ **Status:** ■ NOT STARTED

Phase 3.6 — `src/wdsp/TxChannel.cpp` *implement body*

■ Pre-write: re-read `wdsp/TXA.c::create_txa + xtxa` execution order twice

■ Pre-write: re-read `cmaster.c::XCMSetTX*` family twice

■ Implement: OpenChannel (type=1) + initial setter sequence per reference `_apply_init_setters` pattern (avoiding the §15.23 SetTXABandpassRun trap that hit Lyra-Python)

■ Build + Audit #1 + Audit #2

■ Bench-gate: TUN tone produces clean carrier; SSB voice produces correct sideband

■ **Status:** ■ NOT STARTED

■ PROGRESS TRACKING — STAGE B (aamix.c port) [SIDE-TRACK]

Side-track to Step 14 / Phase 3. The Stage B series ports WDSP's `aamix.c` (RX audio mixer dispatcher) into Lyra-cpp's `src/wdsp/AAMix.{h,cpp}` and wires it into the RX audio path via `WdspEngine::openRx1`. Operates on the same tx-rebuild branch but is independent of Step 14's wire-layer scope.

The arc was operator-driven on a smallest-revertable-step methodology: each sub-stage one focused commit, each gated on the operator's HL2+ bench (audible RX through the AK4951 jack or PC sound card, no clicks/pops, no thread starvation).

Stage B.0 — *resample dependency link check*

■ Verify `create_resample/destroy_resample/flush_resample/xresample` symbols exported by bundled `wdsp.dll`

■ Add `cdef` typedefs to `wdsp_native.{h,cpp}` (4 functions)

■ Shipped 6938022

■ **Status:** ■ COMPLETE

Stage B.1 — *AAMix.h header port*

■ Verbatim port of `aamix.c::AAMix` struct (538 lines, full GPL v3+ attribution to NR0V/WDSP)

■ Plus `Resample.h` (88-line replicated `resample` struct — needed because WDSP accessor setters are extern only, not `__declspec(dllexport)`)

■ 14 public function declarations matching `aamix.h`

■ Single documented [Lyra-native] divergence: `WdspNative*` field for DLL handle plumbing

■ Shipped 20b1df5

■ **Status:** ■ COMPLETE

Stage B.2 — *constructor + destructor + slew helpers*

■ `create_aaslew, destroy_aaslew, flush_aaslew, create_aamix, destroy_aamix`

■ Idiom translations documented: `CRITICAL_SECTION` → `std::mutex`, `HANDLE` semaphore → `std::counting_semaphore<1000>`, `_Interlocked*` → `std::atomic` with `fetch_or/fetch_and`, `malloc0` → `std::vector` RAIL

■ Shipped 427304c

■ **Status:** ■ COMPLETE

Stage B.3 — xMixAudio + xaamix pump + slew + flush_mix_ring

■ Producer-side ring fill via xMixAudio with cs_in[stream] lock

■ Consumer pump xaamix with cs_out lock

■ upslew / downslew cosine envelope helpers

■ flush_mix_ring ring-buffer reset

■ Shipped ca3600c

■ **Status:** ■ COMPLETE

Stage B.4 — close_mixer + open_mixer + mix_main + start_mixthread

■ mix_main consumer thread (std::jthread) with MMCSS Pro Audio priority

■ AND-wait semaphore acquisition (waitForMultipleObjects(N, TRUE, INFINITE) → sequential acquire() loop)

■ close_mixer / open_mixer slew-shutdown / slew-startup atom

■ start_mixthread invocation moved into open_mixer (matches reference flow)

■ Shipped 80d934b

■ **Status:** ■ COMPLETE

Stage B.5 — 10 property setters + CMaster wire-up

■ All 10 SetAAudioMix* / SetAAudioRing* / SetAAudioOutRate / SetAAudioStreamRate setters

■ CMaster.cpp::SendpOutboundRx wired to invoke SetAAudioMixOutputPointer(nullptr, 0, pcm->OutboundRx) (replaces the Stage A PENDING comment)

■ Shipped f052398

■ **Status:** ■ COMPLETE (this is the wire-up site that B.6.b-fix1 later untangled — see B.6.b-fix1 below)

Stage B.6.a — feedIq audio tail refactored into dispatchAudioFrame helper

■ Lines 2402-2456 of wdsp_engine.cpp::feedIq extracted into new WdspEngine::dispatchAudioFrame(audio, nframes) member function

■ Byte-identical relocation — feedIq now calls dispatchAudioFrame inline; behavior unchanged

■ Shipped 27ac2fa

■ Bench-confirmed byte-identical to pre-refactor

■ **Status:** ■ COMPLETE

Stage B.6.b — wire AAMix(1-input passthrough) into RX path

■ **First attempt** (525c2f0) — create_aamix(active=0x01) shortcut. REVERTED at bench (NO audio on either sink). Revert 6e773a5.

■ **Retry** (53d9e5a) — reference-faithful initialization per cmaster.c:297-313: create_aamix(active=0L, ring_size=4096, slew=0/10/0/10ms) → SetAAudioMixOutputPointer(aaMix_, 0, outbound) → SetAAudioMixState(aaMix_, 0, 0, 1) (activate via close_mixer/open_mixer slew atom). STILL NO audio at bench.

■ **Debug-chain instrumentation** (225518c) — 9 atomic counters + ENTRY beacon + producer/consumer 1-Hz log emits. Proved AAMix runs end-to-end (xmix=rel=wake=out=188/s, nzOut=99%, mixmain_started=1). Symptom localized: chain healthy → Outbound is fired 188x/sec → but no audible RX → bug must be downstream of outbound.

■ **Debug-dispatch instrumentation** (12369bc) — 6 atomic counters in dispatchAudioFrame + 1-Hz log. CRITICAL FINDING: [disp-dbg] lines NEVER appeared in the bench log. dispatchAudioFrame was never entered. The outbound callable installed in AAMix was NOT the lambda WdspEngine wired.

■ Stage B.6.b-fix1 (the actual fix, 8b8e0da) — Root cause traced from the timestamps:

- 19:22:02.972 — `openRx1::create_aamix(id=0, ..., Outbound=dispatchAudioFrame_lambda) → paamix[0]->Outbound = real lambda`
- 19:22:02.973 — `HL2Stream::open() → create_rnet() → SendpOutboundRx(STUB) → SetAAudioMixOutputPointer(nullptr, 0, STUB) resolves nullptr → paamix[0] → aaMix_->Outbound = STUB, real lambda CLOBBERED`
- The Stage-A NO-OP stub registration at `RadioNet.cpp:272-283` (its comment: "Stage B aamix port wires it via `SetAAudioMixOutputPointer`") was a placeholder for the wire-up Stage B was supposed to replace with something useful. Stage B.6.b correctly wired `dispatchAudioFrame` in `openRx1` — but left the Stage-A stub registration in place, which then overwrote B.6.b's wiring 1 ms later when `HL2Stream::open()` called `create_rnet()`.
- **Fix:** delete the Stage-A stub registration. The wire-up the Stage-A comment promised now lives in `wdspEngine::openRx1` where the per-channel sink-routing context (`hl2Out_`, `hl2AudioPush_`, `audioRing_`) lives — the global `pcm` cannot reach it. Comment block on the deleted case warns future hands.

■ Stage B.6.b-final (533b06b) — strip diagnostic scaffolding. -134 lines. Brief comment block in `AAMix.cpp` points future regression-hunters at commit history (`git show 225518c` for chain instrument; `git show 12369bc` for dispatch instrument).

■ Operator HL2+ bench validated end-to-end 19:27:49 - 19:28:47 (58-second run):

- HL2 jack: first `[disp-dbg]` after unmute showed `calls=188 muted=0 gain=0.0845 peak16=32767 hl2(push)=188/0 hl2pushSet=1` — audio reaching AK4951
- Operator-confirmed audible RX ("Yes we have audio now")
- Mute toggle worked through dispatch (muted counter 0→22→133→287)
- Output device switch HL2 jack → PC Soundcard at 19:28:17 → `hl2Out=0`, `audioRingSet=1`, `pc(push) 176→928`
- Switch back HL2 jack at 19:28:23 → `hl2Out=1`, `hl2(push)` resumed
- AGC mode changes (med/slow/off/fast) clean
- Chain out=N tracks 1:1 with dispatch `calls=N` across both sinks — no dropped frames

■ Tasks #130 (parent) + #132 (Stage B.6.b) closed

■ Status: ■ COMPLETE — Stage B aamix.c port arc DONE

Stage B+ — next port target TBD

• Operator's call. Candidates per the rolling Stage agenda:

- **Stage C:** `ilv.c` port (TX I/Q interleaver — pairs with the `SendpOutboundTx` stub already wired in `CMaster.cpp` Stage A)
- **Stage D:** `xcmaster` pump body port
- Or pivot back to Step 14 / Phase 3 on the main wire-layer rebuild

• Methodology lessons recorded for next Stage X (in memory file `project_lyra_cpp_tx.md` and §15.28 retrospective):

- 2-stage instrumentation (chain + dispatch) localized the defect to a single line of code on the 2nd bench. No speculation cycle.
- Operator-empirical rule held: after first B.6.b failure I was tempted to revert the `AAMix` port itself; chain counters proved `AAMix` was correct in isolation, redirecting the dig to the wire-up surface where the bug actually was.
- Hard rule for next port: counters first when the symptom is "nothing happens". Instrumentation cost = trivial vs the alternative (multi-round revert/speculate cycle).
- "Do as Thetis does" continues to hold — the `active=0x01` shortcut attempt cost a revert cycle; the reference's `active=0` then `SetAAudioMixState(activate)` pattern worked.

■ VERIFICATION LOG

Per-stage record of audit runs + operator sign-offs. Newest at top.

Stage	Date	Audit#1 result	Audit#2 result	Operator bench	Operator signed
1	2026-06-06	PASS (today's wire-layer round)	PASS (operator RX bench)	RX clean	✓
1.5 (5 fix commits A-E)	2026-06-06	PASS (4-agent wire-layer audit, 196 CLEAN)	PENDING (gates via Stage 2 first-wire)	—	—
§3.9 reverts (3b7888b/42f66c2/12e7acc/6d0e476)	2026-06-07	PASS (5 §3.9-style sign-offs)	PASS (T/PA real values, dead-key 0.2A draw)	RX+telemetry confirmed	✓
2a (Ep6RecvThread prep, wire-INERT)	2026-06-07	PASS (cite table above)	PASS (zero-delta bench — RX works as before, MOX-TX crash pre-existing unchanged)	RX clean; build clean; Rule 2 clean	✓
2b1 (operator-facing counter accessors, wire-INERT)	2026-06-07	PASS (additive only — 4 free-function accessors at lyra::wire namespace scope mirror reference's SeqError file-scope global at networkproto1.c:26; total/framing/window are Lyra-native operator UX with no reference equivalent, classified ACCEPTABLE per Rule 26)	PASS (zero-delta bench — RX still operational)	RX clean; build clean; Rule 2 clean	✓
STAGE B aamix.c port (B.0 → B.6.b-final)	2026-06-08	PASS — reference-faithful to aamix.c byte-for-byte; idiom translations documented; the B.6.b-fix1 surgery in RadioNet.cpp:272 (delete Stage-A SendpOutboundRx no-op stub that clobbered aaMix_->Outbound) closed the silent-audio root cause	PASS — operator HL2+ bench 19:27:49→19:28:47 (58-sec run): audible RX out AK4951 jack at peak16=32767; mute toggle through dispatch; output device swap HL2↔PC Soundcard works; AGC mode changes clean; chain out=N tracks 1:1 with dispatch calls=N across both sinks	Audible RX confirmed: "Yes we have audio now"	✓

■ NOTES FOR FUTURE SESSIONS

- This file IS the source of truth for current state. Read top-to-bottom at session start.
- Any item with **PENDING SIGN-OFF** blocks the stage that depends on it. Surface immediately.

- Any item marked ■ REVERT TO REFERENCE is a PATCH the operator has rejected sign-off on — it will be reverted as part of the relevant Step 14 stage (see §3.9 registry action-queue table). Do NOT carry rejected divergences forward into shipped code.
- §3.9 items: when operator signs off a NEW (post-2026-06-06) divergence, edit this file with the date + commit hash where the divergence ships, NOT in the §3.9 registry only. The bar is high: Rule 1 + Rule 5 reject the default; sign-off requires the operator to explicitly bless a bench-verified divergence.
- If Claude reads a reference body and finds Lyra deviating in a way the existing plan doesn't address → STOP-AND-ASK item added to the "OPEN STOP-AND-ASK" section above, do NOT proceed.
- Backup bundle naming: `_backups/lyra-YYYY-MM-DD-stage{N}.bundle`.
- Commit message format: `[Stage {N}].[sub]] {title}\n\nReference: {file:line}\nAudit: #1=PASS #2={result}\n\nCo-Authored-By: Claude Opus 4.7 (1M context) .`

End of EXECUTION_PLAN.md. Status as of 2026-06-08 EOD:

Stage B aamix.c port arc COMPLETE + operator-bench-validated (commits 27ac2fa → 533b06b on tx-rebuild; backup `_backups/lyra-cpp-2026-06-08-aamix-stage-B-COMPLETE.bundle`). RX audio flows through the ported AAMix dispatcher into both HL2-jack and PC-soundcard sinks; operator confirmed audible. The B.6.b root cause was the Stage-A `SendpOutboundRx` no-op stub in `RadioNet.cpp`:272 clobbering `aaMix->Outbound` ~1 ms after `WdspEngine::openRx1` wired the real lambda — deleted by 8b8e0da. 2-stage diagnostic instrumentation (225518c chain + 12369bc dispatch) localized the defect to one line of code on the 2nd bench; both instrumentation layers stripped in 533b06b. Tasks #130 (parent) + #132 (Stage B.6.b) closed.

Next port target = OPERATOR'S CALL. Stage B side-track done; resume tomorrow on operator direction. Candidates: Stage C `ilv.c` (TX I/Q interleaver — pairs with Stage A's `SendpOutboundTx` stub), Stage D `xcmaster` pump body, or pivot back to Step 14 / Phase 3 wire-layer rebuild on the main arc.

Step 14 status carried forward from 2026-06-07 EOD:

Stage 2a SHIPPED (commit 39f3fa9 — `Ep6RecvThread` reference-position init-sem release + HW-PTT sink scaffolding). Stage 2b1 SHIPPED — operator-facing counter accessors at `lyra::wire` namespace scope (additive, wire-INERT, operator-confirmed zero-delta).

Stage 2b2 SHIPPED (uncommitted, 2026-06-07 EOD; backup `_backups/lyra-cpp-2026-06-07-stage2b2b-ref-faithful.bundle`) — the strict-reference strip-out + reference-faithfulness follow-up pass:

- **Wire-LIVE EP6 migration** — `HL2Stream::rxWorker_jthread` + `rxWorkerLoop` body retired (376-line sed delete); `Ep6RecvThread` is now the sole live EP6 rcv path, wired in `HL2Stream::open()` via `set_router(router_instance(0), 0) + ep6Thread_.start(socket_)`.
- **TX writer migration** — `txWorker_::sendDatagram` now calls `lyra::wire::metis_write_frame(0x02, ...)` (shared `g_mmetis_out_seq_num` — load-bearing for v0.3 PureSignal calcc/iqc per §15.26-class PS-entanglement discipline); the OLD `::sendto(s, ...) + txSeq_.fetch_add` path retired.
- **13+ Lyra-native duplicate atomics DELETED from HL2Stream:** `rxWorker_`, `iqSink_`, `micConsumer_/micConsumerMtx_`, `txSeq_`, `telExciterRaw_/telFwdRaw_/telRevRaw_/telUserAdc0Raw_`, `adcOverloadNow_`, `lastPttIn_`, `micDecimationFactor_/micDecimationCount_`, `totalDg_/seqErrors_/framingErrors_/windowDg_`, `rx1DbFs_/micDbFs_` (+ `rx1DbFs Q_PROPERTY` + accessor).
- **Telemetry accessors** (`hl2TempC` / `paCurrentA` / `fwdPowerW` / `revPowerW`) read `prn->...` direct — sub-decision (a) (reference posture: `prn` fields are the single state-of-record).
- **Counter accessors** route to `lyra::wire::ep6_total_datagrams()` / `ep6_seq_errors()` / `ep6_framing_errors()` / `ep6_drain_window_datagrams()` facades.
- **Auto-LNA** reads `lyra::wire::prn->adc[0].adc_overload` direct (HL2 single-frame-assignment semantic per §3.8 L-3, NOT generic OR-until-cleared).
- `setSampleRate` writes `lyra::wire::mic_decimation_factor = micDec` (plain int per reference, BSS-zero default).
- **TciServer migrated** to `WdspEngine->audioDbFs()` — the reference-faithful WDSP `RXA_S_PK` path.

Mid-flight regression caught + fixed (one round-trip): Router was reference-faithful scaffolding declared but NEVER INSTANTIATED — `router_instance(0)` returned `nullptr`, `register_sink(nullptr, ...)` silently no-op'd, `xrouter(nullptr, ...)` silently early-returned ⇒ RX came up dead. Fix: lifecycle made explicit via reference-shape free functions (see

"Reference-faithfulness follow-up" below).

Reference-faithfulness follow-up pass (4 fixes, operator-directed after the first verify pass flagged 3 ACCEPTABLE-DEVIATIONS + the autoLnaTimer ordering — operator's lock: DEFAULT = fix to match reference, do NOT patch or rationalize):

1. **Router process-singleton via explicit** `create_router(0) / destroy_router(0, 0)` matching `cmaster.c:316` shape. `lyra::wire::create_router(0)` called in `main.cpp` BEFORE `HL2Stream` construction; `destroy_router(0, 0)` in `aboutToQuit` handler-3 AFTER handler-2's `stream->close()`. The transient `HL2Stream::router0_` RAII member (first-fix attempt) retired — RAII-on-`HL2Stream` was a Lyra-native idiom translation; process-lifetime is what the reference does.
2. **HW-PTT pull-model** — new `HL2Stream::onHwPttPoll()` Qt-main-thread slot fired by `hwPttTimer_` at 50 ms / 20 Hz; reads `lyra::wire::prn->ptt_in` direct, edge-detects in `lastHwPttLevel_` (Qt-main sole owner). `Ep6HwPttSink` type + `Ep6RecvThread::set_hw_ptt_sink` setter + `hw_ptt_sink_` member RETIRED (reference has no wire-side push callback — FSM polls `prn->ptt_in` on its own clock). The unconditional wire-side `p->ptt_in = cc[0] & 0x01` write at `Ep6RecvThread.cpp:866` PRESERVED (load-bearing for the new pull-model).
3. `setIqSink` **shim retired** — `main.cpp:252` now calls `lyra::wire::register_sink(router_instance(0), 0, 0, 0, [])(int n, const double* iq){...})` directly with reference `(n, iq)` argument order. No `HL2Stream` surface; matches reference's `LoadRouterAll(prouter[0], ...)` flattened to single-sink case.
4. **Timer ordering** — `statsTimer_ + autoLnaTimer_ + hwPttTimer_.start()` calls MOVED to the very END of `HL2Stream::open()` (after `create_rnet()` allocates `prn` AND `ep6Thread_.start()` brings producer up). Matches `StartAudioNative` (`netInterface.c:32-94`) "allocate state, start producers, then start consumers" pattern. The pre-fix race window where a timer could fire pre-`prn-init` (papered over by a defensive `nullptr` guard) is structurally eliminated.

Final-state audit (2-agent, parallel, 2026-06-07 EOD): **GREEN** — all 4 fixes REFERENCE-FAITHFUL, no new deviations introduced, no orphaned references to retired symbols anywhere in the tree, no Router-class scaffolding-never-instantiated risks remaining, Auto-LNA + HW-PTT timer races structurally unreachable. Build + link clean. RX operator-confirmed working.

Known tracked caveat: `setMicConsumer(...)` on `HL2Stream` is now a documented NO-OP compile stub. `HL2Ep6MicSource` ctor/dtor (`mic_source.cpp:15/:45`) call it → forwarder silently DROPPED ⇒ **TX mic feed does NOT reach WDSP TXA**. Intentional staged-migration interim per `hl2_stream.h:616-627`. **Stage 2b2c (NEXT) = migrate** `mic_source.cpp` to `stream.ep6Thread().set_mic_sink(...)` **direct** with double→float conversion + drop Q6.5 RMS computation; delete the no-op stub once the call sites are gone.

Stage 14.2b2c — mic_source.cpp migration to `Ep6RecvThread::set_mic_sink` (**SHIPPED 2026-06-08, commit 0974b62**)

The Stage 2b2b caveat above is now CLOSED. The mic path runs reference-faithful end-to-end — doubles all the way from `Ep6RecvThread`'s mic-tap through `HL2Ep6MicSource::Consumer` into whatever TX DSP consumer eventually registers via `setConsumer`, matching the reference's `Inbound(inid(1,0), n, prn->TxReadBufp) → CMB ring → cmaster pump → fexchange0(chid(stream,0), pcm->in[stream] /*double*/, ...)` flow. No float bridge anywhere.

Re-verify-twice methodology (operator-directed: "revisit Thetis the reference- read twice-verify first please"):

- **Read #1** captured the reference mic-decode + delivery shape from `networkproto1.c:395-413` (generic) and `:555-579` (HL2 `nddc=4`) — both byte-identical: `bptr[k+0]<<24 | bptr[k+1]<<16 × const_1_div_2147483648_` (`network.h:444, = 1.0/2^31`) → double in `[-1, +1]`; written as interleaved `{TxReadBufp[2i+0]=mic, TxReadBufp[2i+1]=0.0}`; delivered via `Inbound(inid(1,0), mic_sample_count, prn->TxReadBufp)` once per datagram when `mic_sample_count > 0`. `TxReadBufp` is `calloc(2 * sizeof(double) * 720)` (`netInterface.c:1604`) — 720 mic-samples max.
- **Read #2** verified the downstream double-flow: `Inbound(int id, int nsamples, double* in)` at `cmbuffs.c:89-121` writes in into the CMB ring's `r1_baseptr` (interleaved double pairs, copied via `nsamples * sizeof(complex)`), releases `Sem_BuffReady`; the `cmaster` pump thread drains it and calls `fexchange0(chid(stream, 0), pcm->in[stream], ...)` (`cmaster.c:389`); `pcm->in[i]` is `malloc(... * sizeof(complex))` doubles (`cmaster.c:285`). `WDSP fexchange0` takes doubles in and out on TX. The reference NEVER converts to float on the mic path.

Stage 2b2c implements that posture verbatim:

1. `mic_source.h` — `Consumer = std::function`, byte-shape-identical to `Ep6MicSink` (`wire/Ep6RecvThread.h:84-85`) and the reference `Inbound` contract. No float anywhere on the mic path.
2. `mic_source.cpp` — ctor calls `stream_.ep6Thread().set_mic_sink([this](int n, const double* iq){ if (consumer_) consumer_(n, iq); })`. Pure passthrough — no buffer allocation, no double→float conversion, no Q6.5 RMS bench instrument,

no Task #40 safetyLog qWarnings (none of which exist in the reference path). Dtor clears with `stream_.ep6Thread().set_mic_sink({})` under the discipline documented at `Ep6RecvThread.cpp:305-311` ("Releasing a sink via `set_*({})` from a destructor is permitted ONLY after the EP6 thread is joined").

3. `hl2_stream.h` — the no-op `setMicConsumer(...)` compile stub deleted entirely. Private-section commentary referencing `micConsumer_ / micConsumerMtx_ / the csIN-equivalent / Q6.5 staging retired in favour of a one-paragraph pointer at the new mechanism + RadioNet.cpp's TU-scope mic_decimation_factor.`
4. `main.cpp` — `aboutToQuit` handler ordering reworked so the F1 set-once-before-start contract holds at teardown. The old `delete micSource` step lived inside `handler-1` (which fired BEFORE `stream->close()` — that worked for the prior mutex-blocking `setMicConsumer({})` shim but breaks against the new `assert_not_running`). New order: `handler-1` (TX cleanup) → `handler-2` (`stream->close()` → joins `ep6Thread`) → **handler-3 (new: delete `micSource`)** → `handler-4` (`destroy_router(0,0)`, was `handler-3`, log strings updated). Qt fires `aboutToQuit` slots in registration order, so `handler-3's ~Hl2Ep6MicSource` → `ep6Thread().set_mic_sink({})` lands with `ep6Thread_` provably joined and the captured this in the registered lambda safe by construction.

Build: 31/31 ninja green, `lyra.exe` links clean. Two warnings (`bandmemory.cpp:109` C4702 unreachable, `hl2_stream.cpp:776` C4456 err shadowing) are pre-existing.

Reference parity self-audit: GREEN. Doubles end-to-end on the mic path; signature match against `Ep6MicSink + Inbound`; lambda body pure passthrough; F1 contract honoured at both endpoints (ctor pre-start, dtor post-join); no orphan `setMicConsumer` callers; no leftover float-Consumer references; stale commentary retargeted.

Backup: `_backups/lyra-cpp-2026-06-08-stage2b2c.bundle` (full git bundle, 15.9 MB).

Downstream still pending: `Hl2Ep6MicSource::setConsumer(...)` has no live caller — the `TxDspWorker` that would feed mic doubles into the WDSP TXA chain was removed in TX-rip Phase 1 (Q2) and is being rebuilt per `docs/TX_ARCHITECTURAL_MAPPING.md §10.3`. Stage 2b2c is the wire-side half: the mic path is now correctly wired all the way to the operator-facing Consumer boundary. Wiring the TX DSP consumer + the wire-LIVE TX path is the next stage.

Stage 14.2b2d — `lyra::wdsp::TxChannel` *body, wire-inert (SHIPPED 2026-06-08)*

Phase 1 empty `TxChannel.h/cpp` shells get a reference-faithful 1:1 body per §10.3 item 4. Wire-inert: nothing constructs the class yet, nothing calls its methods, runtime behaviour is byte-identical to Stage 2b2c.

Honest correction note: the initial Stage 2b2d ship (commits `e995740 + 0265d02`) contained three real reference deviations caught by the operator's "no patches" challenge and corrected before re-shipping:

1. `cfg.inSize` default 512 was a number I invented — the reference value is `getbufsize(48000) = 64 (cmsetup.c:106-111 + :71)`. **Fixed:** `inSize` removed from `TxConfig` entirely; computed at `open()` time via `referenceBufferSize(inRate) = 64 * rate / 48000` matching reference exactly.
2. `computeOutSize` rounded up to the next power-of-2, which the reference's `getbufsize` does NOT do. I had even written a comment claiming the reference rounded — false. **Fixed:** `computeOutSize` deleted; output buffer sized to `referenceBufferSize(outRate)` matching reference `cmsetup.c:84`.
3. `stop(bool blocking = true)` exposed a non-blocking option for which the reference has zero call-site precedent (both `console.cs:30355` `keyup` and `destroy_xmtr cmaster.c:265` unconditionally use `dmode=1`). **Fixed:** parameter removed; `stop()` always issues blocking-flush.

The reset + re-ship preserves a clean single-commit landing for Stage 2b2d in reference-faithful form; the earlier patched-state commits are retained in the prior backup bundle for archaeology.

The shipped class wraps the canonical WDSP TXA channel lifecycle:

- `open()` invokes `OpenChannel(channel, inSize_, 4096, 48000, 96000, 48000, type=1, state=0, 0.000/0.010/0.000/0.010, block=1)` — every parameter matches `cmaster.c:177-190` byte-for-byte. `inSize_ = referenceBufferSize(inRate) = 64` for HL2 48 kHz mic.
- Three TX output buffers (`out[0]/out[1]/out[2]`) allocated to `referenceBufferSize(outRate)` pairs unconditionally at construction per `cmaster.c:126-127` reference posture. `out[1]` (EER) and `out[2]` (sidetone) sit unused until their helpers land in their own queued stages — but the buffers are held per reference.
- `close()` calls `CloseChannel(ch) alone` — matches `destroy_xmtr (cmaster.c:255-271)` verbatim. The reference at line 265 is literally `CloseChannel (chid (inid (1, i), 0));` with NO preceding `SetChannelState`. The blocking-flush `SetChannelState(0,1)` discipline belongs at `keyup (stop())`; reference `console.cs:30355`, NOT at destroy time. The initial

Stage 2b2d ship of `close()` conflated keyup discipline + destroy discipline into one method; caught by the deeper PASS 1 reference cross-check below.

- `start()` wraps `SetChannelState(ch, 1, 0)` matching `console.cs:30346`. `stop()` wraps `SetChannelState(ch, 0, 1)` — always blocking, no parameter, no non-reference variant.
- `setMode(TxaMode)` wraps `SetTXAMode` — `TxaMode` enum mirrors `wdsp/TXA.h:31-47` enum `txaMode 1:1 (LSB=0 .. AM_USB=13)`.
- `setBandpass(lo, hi)` wraps `SetTXABandpassFreqs`. **Documented LANDMINE:** never calls `SetTXABandpassRun` (the §15.23 root cause — `bp1` toggle, not `bp0` sideband select).
- `process(mic_iq, n_samples)` wraps `fexchange0(channel, mic_iq, outBuf_, &err;)` matching `cmaster.c:389` TX dispatch. `const_cast` at the boundary is the standard Rule 26 C-to-C++23 const-correctness bridge (WDSP `cdef` takes non-const `double*` for historical C reasons; only READS through the input pointer in practice).

Idiom translations (Rule 26 — all reviewed against the strict-reference rule and justified):

- C++23 class `TxChannel` wraps the reference's C `pcm->xmtr[i]` struct + free-function pattern. Same idiom precedent as `RxChannel` (operator-approved when that shipped).
- `std::vector` for the three output buffers replaces reference `malloc(2*sizeof(complex)*ch_outsize)`. Bits passed to `fexchange0` are identical; ownership idiom is C++23-native.
- `const double*` on `process()` arg matches Stage 2b2c's `HL2Ep6MicSource::Consumer` signature; `const_cast` at the WDSP boundary is the standard C-to-C++23 const bridge.
- enum class `TxaMode` is C++23 strong-typed mirror of the C enum `txaMode` — same integer values, namespace-scoped.
- `if (!opened_) return -1;` RAII state guard on `process()` — Rule 26 C++23 defensive API idiom. The reference assumes caller has set the channel up; the guard makes that explicit at the boundary. Behaviour is byte-identical to bare `fexchange0` when inputs are valid.
- `constexpr int referenceBuffsize(int rate)` noexcept helper — namespace-scoped C++23 mirror of reference `getbuffsize(rate) = 64 * rate / 48000` (`cmsetup.c:106-111`). Same return for every input.

Audit-follow-up note (deeper PASS 1 reference cross-check, 2026-06-08): after the Stage 2b2d soft-reset re-ship, an operator-directed line-by-line audit re-reading each cited reference file (`cmsetup.c:106-111`, `TXA.h:31-47`, `cmaster.c:112-271`, `cmaster.c:389`, `TXA.c:827+`, `cmbuffs.c:89`, `networkproto1.c:395-413/560-579`, `bandpass.c:466`) caught one additional real deviation + one doc-accuracy nit, both fixed in this commit:

1. `close()` **had** `SetChannelState(ch, 0, 1)` **before** `CloseChannel`. The cited justification claimed this matched "`console.cs:30355` keyup discipline + `destroy_xmtr` (`cmaster.c:265`)" — but those are two SEPARATE reference sites used in two SEPARATE situations: keyup is operator-PTT-release (channel stays OPEN), destroy is teardown (channel goes AWAY). Reference `destroy_xmtr` calls `CloseChannel` ALONE at line 265, with no preceding `SetChannelState`. **Fixed:** removed the `SetChannelState(0,1)` from `close()`; `stop()` (the keyup analog) keeps it. The two reference sites are now correctly separated in Lyra: `stop()` ↔ `keyup` (`console.cs:30355`), `close()`/dtor ↔ `destroy_xmtr` (`cmaster.c:265`).
2. `setMode` **comment cited** "`TXA.c:823-829`" — that line range is actually `TXAUslewCheck` (an unrelated function); `TXASetupBPFilters` begins at `TXA.c:827+`. **Fixed:** comment updated to "`TXA.c:827+`". No code change; doc accuracy only.

`SetTXABandpassRun` zero call sites + symbol absent from `cdef` confirmed by `grep` across the entire Lyra-cpp tree (also confirmed in reference: defined at `bandpass.c:466`, called nowhere else in Thetis). LANDMINE-protected at both the comment layer AND the linker layer.

Explicitly deferred to later queued stages per §10.3 (each ships in its own reference-verify-twice / no-patch cycle):

- `create_dexp` (VOX)
- `create_aamix` (anti-VOX mixer)
- `create_txgain` (Penelope / PS gain — note: HL2 uses different mechanism per CLAUDE.md §3.8 step-attenuator + `drive_level`; that's protocol-layer, not WDSP-layer)
- `create_eer` / `create_ilv` (EER + EER interleaver — operator DB has EER off)
- `create_sidetone` (CW sidetone — v0.2.2)
- `XCreateAnalyzer` (TX-side panadapter source — polish stage)

Build: ninja green, `lyra.exe` links clean. Zero new warnings.

No hardware bench required at this stage — the class is dead code at runtime. Hardware verification gate moves to Stage 2b2e (when something constructs TxChannel + registers it on micSource->setConsumer(. . .)).

Operator-directed next checkpoint: detailed Reference-vs-Lyra TX audit BEFORE proceeding to Stage 2b2e. Sweep the entire TX surface (FrameComposer + PttFsm + TxChannel + Ep2SendThread integration plan + ATT-on-TX + PA-enable + mic source plumbing + any other §10.3 Phase 1 items) against the Thetis reference, flag deviations, present findings BEFORE any further code.

Operator hardware bench pending (NOT run this session — code-level audits only): cold open RX1 audio + panadapter, telemetry T/V/PA, Auto-LNA on a real signal, ×5 stop/restart, HW-PTT-opt-in foot-switch (after per-unit ptt_in-at-rest bench-verify per §10 Q#1), MOX bit flip without PA.

Methodology discipline (operator-locked 2026-06-07, REAFFIRMED EOD): DEFAULT VERDICT = strict reference. Lyra-native preservation is NEVER the default answer. **Default action on any flagged deviation = fix to match reference. NOT patch. NOT deviate. NOT rationalize "acceptable."** Verify-pass output gets re-judged against this rule before presentation; agents report what they find, the rule decides what gets actioned. RX options (WDSP NR/AGC/ANF/LMS/NB/SQ/AEPF/captured profile/etc.) STAY (WDSP-backed = reference-equivalent). TX operator-feature suite (Combinator, Plate Reverb, EQ, CFC, speech-enhancement, mic profiles) is where Lyra deliberately differs — strict-reference rule does NOT auto-apply there; WE TALK before any change near those. LNA — flag if anything LNA-related interacts with TX path; WE TALK.**